Lecture 10 — Mock Regional

> Regional Level · Prerequisites: Lectures 1–9 of this track · Estimated time: 5 hours contest + 1.5 hours debrief

1. Overview

This lecture is a full mock regional: eleven original problems (A–K) modelled on the shape of a real ICPC regional set, followed by a structured debrief. Every lecture of the track appears at least once, in disguise. The coach prints Section 3 (statements) for the contest and keeps Sections 4–5 (solutions, reference code) for the debrief.

The statements below are summaries; before running the mock, the coach should expand each into a full statement with input/output format and samples (the reference implementations in Section 5 and the sketches in Section 4 make generating sample data straightforward). Time limits: 2 seconds for every problem.

2. Running the mock

- **Rules**: 5 hours, teams of three, one computer, printed notebook, standard ICPC scoring (solved problems, then penalty with 20 minutes per rejected submission on solved problems), scoreboard frozen for the final hour.
- **Judge**: any local judge (DOMjudge, a Codeforces Gym mashup with your own tests, or the coach checking outputs by script).
- **Order**: problems are alphabetical, not sorted by difficulty, exactly as at NWERC or NAC.
- **Logging**: each team keeps the problem board from Lecture 1. The board is the main input to the debrief, so it must record when each problem was read, labelled, started, and accepted.

3. Problem statements (summaries)

A — Odometer Without Fours. A faulty odometer never displays a number containing the digit 4: after 3 it shows 5, after 39 it shows 50. It started at 0. Given the displayed value D (no digit 4, $0 \le D \le 10^{18}$), output the true distance driven.

B — Arrow Grid. An $R \times C$ grid ($R, C \le 1000$) has an arrow `U`, `D`, `L`, or `R` in each cell. A robot starts at a given cell and every second moves one cell in the direction of the arrow it is standing on; if that move would leave the grid, it stays where it is forever. Output its cell after $K \le 10^{18}$ seconds.

C — Toll Vouchers. A road network has $n \le 10^5$ cities and $m \le 2 \cdot 10^5$ one-way roads with tolls up to 10^9. You have $k \le 10$ vouchers; each makes one road free. Find the minimum total toll from city 1 to city n, or report that n is unreachable.

D — Lab Bookings. One laboratory, $n \le 2 \cdot 10^5$ experiments. Experiment i takes p_i hours and must be finished by hour d_i ($p_i, d_i \le 10^9$). Experiments run one at a time without interruption starting at hour 0. Maximise the number of experiments completed on time.

E — Double Coverage. Given $n \le 2 \cdot 10^5$ half-open intervals $[l_i, r_i)$ with $-10^9 \le l_i < r_i \le 10^9$ and an integer K, output the total length of the set of points covered by at least K intervals.

F — Gears. There are $n \le 1000$ gears; gear i has $m_i \le 10^9$ teeth, and its marked tooth is currently at position r_i ($0 \le r_i < m_i$). Every second each gear advances by one tooth (position r_i becomes $r_i + 1 \bmod m_i$). Output the first second $t \ge 0$ at which every marked tooth is at position 0, or `never`. The lcm of all m_i is guaranteed to be at most 10^{18}.

G — Palindrome Banner. Given a lowercase string s ($|s| \le 10^6$), append as few letters as possible to its end so that it becomes a palindrome. Output the resulting palindrome.

H — Rest Stops. A hiking trail has checkpoints $1 \dots n$ ($n \le 10^6$); resting at checkpoint i costs $c_i \ge 0$. You start at checkpoint 0 and finish at $n + 1$, both free. Between any two consecutive rests (counting start and finish) you may pass at most $k - 1$ checkpoints, i.e. their indices differ by at most k. Minimise the total cost.

I — Coprime Committees. A club has $n \le 2 \cdot 10^5$ members with ratings $a_i \le 10^6$. A committee is a non-empty subset of members; it is *balanced* if the gcd of its ratings is 1. Count balanced committees modulo $10^9 + 7$.

J — Fence Lattice. $n \le 10^5$ trees stand at distinct integer points with coordinates of absolute value at most 10^6. A fence is built along the convex hull of the trees. How many lattice points lie strictly inside the fence?

K — Forbidden Jingle. Count the strings of length $n \le 10^{18}$ over the first $\sigma \le 26$ lowercase letters that do not contain the string p ($1 \le |p| \le 30$) as a contiguous substring, modulo $10^9 + 7$.

4. Solutions and debrief notes

Problem	Difficulty	Technique	Lecture
A	Easy	base 9	2
E	Easy	1D sweep	6
B	Easy-medium	cycle detection	2

Problem	Difficulty	Technique	Lecture
C	Medium	Dijkstra on layered states	4
D	Medium	Moore–Hodgson	3
F	Medium	general CRT	7
G	Medium	prefix function	9
H	Medium	DP + monotonic deque	5
J	Medium	convex hull + Pick	8
I	Medium-hard	exact-gcd sieve	7
K	Hard	automaton + matrix power	9

A. The displayed values are exactly the non-negative integers without the digit 4, in increasing order, so the true distance is the index of `D` in that sequence. Such numbers are base-9 numerals written with the digit set {0, 1, 2, 3, 5, 6, 7, 8, 9}. Map 5–9 to 4–8 and read the result in base 9. Example: display 15 → digits (1, 4) in base 9 → 13 (the skipped values were 4 and 14). Answers reach about $1.35 \cdot 10^{18}$, so 64-bit integers are required.

E. Create events `(l, +1)` and `(r, −1)`, sort by coordinate, and sweep; between consecutive event coordinates, add the gap length if the running count is at least `K`. The order of events at equal coordinates does not matter, because the gap between them is zero; a good debrief question is to explain why it *would* matter for "maximum number of overlapping intervals". The total length can reach $2 \cdot 10^9$: use `long long`.

B. The next cell depends only on the current cell, so the state is the cell itself, with `R·C ≤ 10⁶` states. Simulate while recording the first time each cell is visited; at the first repeat, jump ahead using the cycle length, as in Lecture 2. A robot whose arrow points out of the grid is in a fixed point; this is a cycle of length 1, but it must be detected before attempting to move.

C. Layered Dijkstra (Lecture 4) on states `(city, vouchers used)`: about $1.1 \cdot 10^6$ states and $2.2 \cdot 10^6$ edges per layer pair. Using a voucher on road `u → v` moves from layer `j` to `j + 1` with cost 0. The answer is the minimum over all layers, since using fewer than `k` vouchers is allowed. Distances need 64 bits.

D. Moore–Hodgson (Lecture 3): sort by deadline, add each experiment's duration to a running total and to a max-heap, and whenever the total exceeds the current deadline remove the longest experiment so far. The answer is the heap size. Teams should be able to state the invariant (maximum count, minimum total duration among maximum-count feasible sets).

F. The condition is `t ≡ −r_i (mod m_i)` for all `i`. The moduli are arbitrary, so use the general CRT merge from Lecture 7, checking `gcd | (difference)` at each step and multiplying through `__int128`. If all `r_i = 0`, the answer is 0. Normalise `−r_i mod m_i` to `[0, m_i)`.

G. Appending letters to the end can only help by keeping a palindromic suffix of `s` in the middle of the final string. The answer is `s` followed by the reverse of `s[0..n−L)`, where `L` is the length of the longest palindromic suffix. Compute `L` as the last value of the prefix function of `rev(s) + '#' + s`, or with Manacher. The separator prevents the border from exceeding `n`.

H. `dp[i] = c_i + min_{i−k ≤ j < i} dp[j]` with `dp[0] = 0` and `c_{n+1} = 0`; the answer is `dp[n+1]`. The window minimum is a monotonic deque (Lecture 5), giving O(n). An O(n log n) segment-tree solution also passes; an O(n·k) loop does not.

J. Build the convex hull (Lecture 8) and apply Pick's theorem, `I = (2A − B + 2)/2`, where `B` is the sum over hull edges of `gcd(|dx|, |dy|)`. Whether collinear points are kept on the hull does not change `B`. If all trees are collinear (or `n ≤ 2`) the fence encloses no area and the answer is 0. Twice the area is at most about $1.6 \cdot 10^{13}$: use `long long`.

I. Let `c_d` be the number of ratings divisible by `d`. Subsets whose gcd is a multiple of `d`: `F(d) = 2^{c_d} − 1`. Subsets with gcd exactly `d`: `G(d) = F(d) − Σ_{k≥2} G(k·d)`, computed for `d` from the maximum value down to 1 (Lecture 7). The answer is `G(1)`. Both harmonic loops are O(V log V).

K. Build the KMP automaton of `p` (Lecture 9) on states `0 … m−1`. Let `T[a][b]` be the number of letters leading from state `a` to state `b` without completing the pattern; transitions to state `m` are simply omitted. The number of valid strings is the sum of row 0 of T^n. Fast exponentiation costs $O(m^3 \log n) \approx 27\,000 \cdot 60$ multiplications. A frequent mistake is to make state `m` absorbing and then sum the wrong entries.

5. Reference implementations

All four programs were checked against brute-force solutions on random small inputs.

5.1 A — Odometer Without Fours

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string d;
    cin >> d;
    unsigned long long v = 0;
    for (char ch : d) {
        int x = ch - '0';
        if (x > 4) x--;                  // digits 5..9 represent 4..8
        v = v * 9 + x;
    }
    cout << v << '\n';
}
```

5.2 H — Rest Stops

Input: `n k`, then `c_1 … c_n`.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<ll> c(n + 2, 0), dp(n + 2, 0);
    for (int i = 1; i <= n; i++) cin >> c[i];
    deque<int> dq;                          // indices with increasing dp values
    dq.push_back(0);
    for (int i = 1; i <= n + 1; i++) {
        while (dq.front() < i - k) dq.pop_front();
        dp[i] = c[i] + dp[dq.front()];
        while (!dq.empty() && dp[dq.back()] >= dp[i]) dq.pop_back();
        dq.push_back(i);
    }
    cout << dp[n + 1] << '\n';
}
```

5.3 I — Coprime Committees

Input: `n`, then `a_1 … a_n`.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1'000'000'007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    int V = 1;
    for (auto& x : a) { cin >> x; V = max(V, x); }
    vector<int> freq(V + 1, 0);
    for (int x : a) freq[x]++;
    vector<ll> pw2(n + 1, 1);
    for (int i = 1; i <= n; i++) pw2[i] = pw2[i - 1] * 2 % MOD;
    vector<ll> G(V + 1, 0);
    for (int d = V; d >= 1; d--) {
        int c = 0;
        for (int m = d; m <= V; m += d) c += freq[m];
```

```
        ll g = (pw2[c] - 1 + MOD) % MOD;
        for (int m = 2 * d; m <= V; m += d) g = (g - G[m] + MOD) % MOD;
        G[d] = g;
    }
    cout << G[1] << '\n';
}
```

5.4 K — Forbidden Jingle

Input: `n σ p`.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1'000'000'007;
using Mat = vector<vector<ll>>;

Mat mul(const Mat& A, const Mat& B) {
    int m = A.size();
    Mat C(m, vector<ll>(m, 0));
    for (int i = 0; i < m; i++)
        for (int k = 0; k < m; k++) {
            if (!A[i][k]) continue;
            for (int j = 0; j < m; j++) C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
        }
    return C;
}

int main() {
    ll n;
    int s;
    string p;
    cin >> n >> s >> p;
    int m = p.size();
    vector<int> pi(m, 0);
    for (int i = 1; i < m; i++) {
        int k = pi[i - 1];
        while (k > 0 && p[i] != p[k]) k = pi[k - 1];
        if (p[i] == p[k]) k++;
        pi[i] = k;
    }
    vector<vector<int>> go(m, vector<int>(s));
    for (int st = 0; st < m; st++)
        for (int c = 0; c < s; c++) {
            if (p[st] - 'a' == c) go[st][c] = st + 1;
            else go[st][c] = (st == 0) ? 0 : go[pi[st - 1]][c];
        }
    Mat T(m, vector<ll>(m, 0)), R(m, vector<ll>(m, 0));
    for (int st = 0; st < m; st++)
        for (int c = 0; c < s; c++)
            if (go[st][c] < m) T[st][go[st][c]]++;
    for (int i = 0; i < m; i++) R[i][i] = 1;
    for (ll e = n; e > 0; e >>= 1) {
```

```
        if (e & 1) R = mul(R, T);
        T = mul(T, T);
    }
    ll ans = 0;
    for (int j = 0; j < m; j++) ans = (ans + R[0][j]) % MOD;
    cout << ans << '\n';
}
```

6. The debrief

Hold the debrief the same day if possible, before anyone looks up solutions. Allow about 90 minutes.

6.1 Timeline table

Each team fills in one row per problem from its problem board:

Problem	Read at	Label	Started	Accepted at	Rejected tries	Time lost on	Root cause
A							
B							
…							

Root causes, one per incident: **misread** statement, **wrong idea**, **implementation bug**, **overflow**, **TLE**, **output format**, **idle machine** (someone thinking at the keyboard), **coordination** (two people on one problem, lost hand-off).

6.2 Questions

1. Compare your solve order with the intended difficulty table. Which problem did you solve latest relative to its difficulty, and why?
2. How many minutes was the machine idle or used for thinking? (Estimate from the board.)
3. Which rejections would the 60-second checklist from Lecture 1 have prevented?
4. Which unsolved problem was closest? Was the missing piece an idea or an implementation?
5. Which notebook snippet was missing or wrong? Which one saved time?
6. One concrete change in team process for the next contest.

6.3 Upsolving

- Within one week, every member solves every problem the team did not solve, alone.
- For every rejected submission, write the test that would have caught it and add it to a personal "trap list".

- Add any missing notebook code (for example, the matrix power from K or the general CRT from F), compiled and tested.

6.4 Interpreting the score

Solved	Interpretation
3 (A, E, B)	Fundamentals in place; focus on Lectures 3–5
5–6	Solid regional team; work on speed and penalty
8–9	Medal contention at most regionals
10–11	Ready for the National Championship track

7. Common mistakes (seen in previous runs of this kind of mock)

- **A**: treating the display as base 10 minus the count of "numbers with 4" computed incorrectly; 32-bit overflow.
- **B**: not handling the "stays forever" state; using `int` for `K`.
- **C**: answering only `dist[n][k]` (exactly `k` vouchers).
- **D**: removing the current experiment when late instead of the longest.
- **E**: `int` lengths.
- **F**: assuming coprime moduli; products overflowing `long long`.
- **G**: prefix function of `rev(s) + s` without separator.
- **H**: forgetting that the finish also needs a rest within distance `k`.
- **I**: forgetting `+ MOD` in subtractions.
- **J**: crashing on collinear input; area as `int`.
- **K**: off-by-one in the matrix dimension; making the forbidden state absorbing.
- **Team level**: nobody reading K until the last hour, or three people stuck on I while H (medium) sat unread.

8. Practice problems: more mocks

Use real regional sets for further mock contests (most are available as Codeforces Gym contests or on Kattis; search for the contest name and year):

- NWERC (Northwestern Europe) — e.g. NWERC 2022 and 2023.
- SWERC (Southwestern Europe).
- CERC (Central Europe).
- ICPC North America Championship and the NA regional contests (NAC, NCNA, SER, ...).
- NCPC (Nordic), a good "first full mock" because of its gentle easy problems.
- ICPC Asia regionals (Jakarta, Seoul, Taichung, ...), often available on Codeforces Gym.

Individual problems used in this track that make good warm-ups before a mock: Codeforces 1B, CSES 1195, Codeforces 1526C2, Codeforces 372C, CSES 1741, Kattis `generalchineseremainder`, CSES 2193, CSES 1112.

9. Further reading

- Lecture 1 of this track (triage, the problem board, the 60-second checklist).
- ICPC regional results and problem archives: https://icpc.global (follow the links to each regional's site).
- DOMjudge (https://www.domjudge.org) for running a local judge with your own test data.
- National Championship Prep track, Lecture 1 (training systems, virtual contests, upsolving), which picks up from this debrief.